Fortunately, Ruby has a standard message, to_s, that it sends to any object it wants to render as a string. The default behavior of to_s, defined in the Object class, is the ClassName then colon then object id behavior we just described, So, when we pass one of our BookInStock objects to puts, the puts method calls to_s in that object to get its string representation.

If we want different behavior, we can override the default implementation of to_s to give us a better rendering of our objects, we'll talk more about how this works in Chapter 6, Sharing Functionality: Inheritance, Modules, and Mixins, on page 101:

tut_classes/stock_stats/book_in_stock_2.rb

```
class BookInStock
  def initialize(isbn, price)
    @isbn  = isbn
    @price = Float(price)
  end

  def to_s
    "ISBN: #{@isbn}, price: #{@price}"
  end
end

b1 = BookInStock.new("isbn1", 3)
puts b1
b2 = BookInStock.new("isbn2", 3.14)
puts b2
b3 = BookInStock.new("isbn3", "5.67")
puts b3
```

produces:

```
ISBN: isbn1, price: 3.0
ISBN: isbn2, price: 3.14
ISBN: isbn3, price: 5.67
```

The p method actually has a different message it calls on objects, that method is named inspect. The difference is that inspect is designed to produce a representation that is useful to a developer when debugging and to_s is supposed to produce a human-readable one for more general output.

There's something going on here that's both trivial and profound. See how the values we set into the instance variables @isbn and @price in the initialize method are subsequently available in the to_s method? That shows how instance variables work—they're stored with each object and available to all the instance methods of those objects.

Objects and Attributes

The BookInStock objects we've created so far have an internal state (the ISBN and price). That state is private to those objects—no other object can access an object's instance variables. In general, this is a Good Thing. It means that the object is solely responsible for maintaining its own consistency. (We feel obligated to note here that there is no such thing as perfect privacy in Ruby, and you should not depend on Ruby's language privacy for security purposes.)

However, an object that is totally secretive is pretty useless—you can create it, but then you can't do anything with it. You'll normally define methods that let you access and manipulate